

Алгоритм Диница

Подготовил Сивухин Никита. По вопросам пишите на почту sivukhin.work+teach@gmail.com

К текущему моменту мы знаем 3 алгоритма поиска максимального потока в сети со следующими асимптотиками:

- Алгоритм Форда-Фалкерсона: $O(|E||f|)$
- Алгоритм Эдмондса-Карпа: $O(|E|^2|V|)$
- Алгоритм масштабирования потока: $O(|E|^2 \log U)$, где $U = \max\{c(u, v) \mid (u, v) \in E\}$

В данной лекции мы разберём алгоритм Диница нахождения максимального потока за время $O(|E||V|^2)$.

Для описания алгоритма Диница нам понадобится ввести определение *блокирующего потока* в транспортной сети $G = (V, E, c)$. Так, поток f в G называется *блокирующим*, если **не существует** такого пути $P = (s = v_0, v_1, \dots, v_k = t)$, что $\forall i : f(v_i, v_{i+1}) < c(v_i, v_{i+1})$. Другими словами, любой путь из истока в сток содержит хотя бы одно полностью насыщенное ребро $(u, v) : f(u, v) = c(u, v)$.

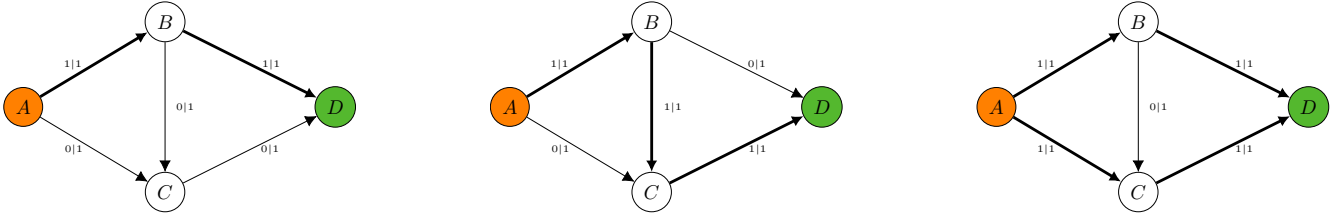


Рис. 1: Пример транспортной сети и потоков в ней. Потоки на второй и третьей картинке являются блокирующим, в то время как поток на первой картинке — нет, потому что вдоль пути (A, C, D) нет ни одного насыщенного ребра.

Определим теперь вспомогательную «слоистую» остаточную сеть G_f^L , как транспортную сеть на основе остаточной сети G_f , состоящую из рёбер (u, v) таких, что $d[v] = d[u] + 1$, где d — расстояние от истока s до вершины v . Таким образом G_f^L состоит из рёбер принадлежащих графу кратчайших путей относительно s в G_f .

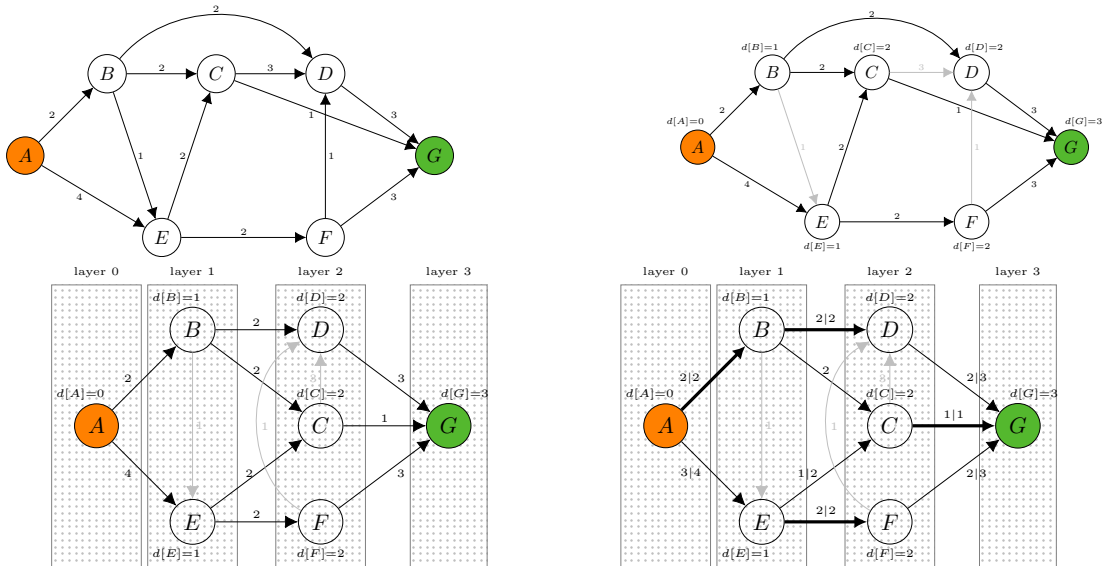


Рис. 2: Пример транспортной сети G и «слоистой» вспомогательной сети G_f^L , в которой участвуют только рёбра из графа кратчайших путей (т.е. такие (u, v) , что $d[v] = d[u] + 1$). На правой нижней картинке изображён блокирующий поток во вспомогательной сети G_f^L .

Тогда, схема алгоритма Диница заключается в том, что на очередной итерации строится вспомогательная «слоистая» сеть G_f^L относительно текущего потока f , после чего в ней находится блокирующий поток f' . Эта процедура повторяется до тех пор, пока в G_f^L существует ненулевой блокирующий поток.

Данную схему алгоритма можно выразить следующим каркасом кода, а позже мы разберем реализацию подробнее.

```

1 struct Graph {
2     int dinic_flow() {
3         int total_flow = 0;
4         while (true) {
5             auto layered = build_layered_residual_net();
6             auto f = find_blocking_flow(layered);
7             if (f == 0) { break; }
8             total_flow += f;
9         }
10        return total_flow;
11    }
12 };

```

Корректность алгоритма Диница следует из того факта, что если в остаточной сети G_f есть дополняющий путь из истока в сток, то во вспомогательной слоистой сети также будет путь из истока в сток, а значит по теореме Форда-Фалкерсона алгоритм найдет максимальный поток, так как в конце своей работы в G_f не будет дополняющих путей.

Покажем, что алгоритм совершает не более $O(|V|)$ внешних итераций цикла (т.е. будет не более $O(|V|)$ операций построения блокирующего потока). Для этого сначала вспомним теорему из лекции об алгоритме Эдмондса-Карпа:

Лемма 1. Если в транспортной сети $G = (V, E)$ увеличение потока происходит вдоль кратчайшего $s \rightsquigarrow t$ пути в остаточной сети G_f , то для всех вершин $v \in V \setminus \{s\}$ длина кратчайшего пути $d_f(s, v)$ в остаточной сети G_f не уменьшается.

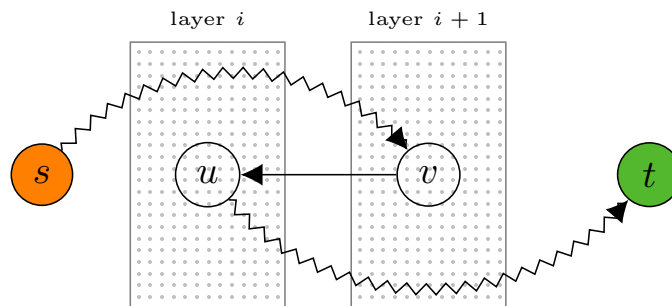
Так как алгоритм Диница всегда увеличивает поток вдоль кратчайших путей, то лемма 1 верна. Докажем, что для стока t верно более строгое утверждение:

Лемма 2. После применения блокирующего потока в G_f^L к потоку в сети G_f расстояние по рёбрам остаточной сети $G_{f'}$ до стока строго увеличивается: $d'[t] > d[t]$.

Доказательство. От противного: допустим что расстояние до стока не изменилось после применения блокирующего потока, т.е. $d'[t] = d[t]$. Заметим, что так как блокирующий поток построен на основе вспомогательной сети G_f^L , то в остаточной сети $G_{f'}$ относительно G_f могли добавиться только рёбра, обратные рёбрам из G_f^L .

Покажем сначала, что любой $s \rightsquigarrow t$ путь P' в $G_{f'}$ проходящий по обратному ребру из G_f^L будет иметь длину хотя бы $d[t] + 2$.

Рассмотрим последнее обратное ребро в пути $P' = (s, \dots, v, u, \dots, t)$, где (u, v) — ребро из G_f^L , а значит $d[v] = d[u] + 1$. Так как (v, u) — последнее обратное ребро в P' , то путь $u \rightsquigarrow t$ содержит только рёбра из G_f . Но тогда можно построить путь P в G_f как кратчайший путь от истока до u , после чего добавить в него суффикс $u \rightsquigarrow t$ пути P' . Такой путь будет иметь длину $d[u] + |u \rightsquigarrow t| \geq d[t]$, однако известно, что длина пути P' не меньше чем $d'[v] + 1 + |u \rightsquigarrow t| \geq d[v] + 1 + |u \rightsquigarrow t| \geq d[u] + 2 + |u \rightsquigarrow t| \geq d[t] + 2$.



Таким образом, если $d'[t] = d[t]$, то кратчайший $s \rightsquigarrow t$ путь в $G_{f'}$ должен состоять только из рёбер остаточной сети G_f , но в таком случае он принадлежит графу кратчайших путей, а значит должен быть заблокирован в G_f^L . $\square$

Таким образом, пользуясь леммой 2 можно заключить, что алгоритм сделает не более $|V| - 1$ итераций внешнего цикла. Осталось разобраться, как быстро искать блокирующий поток во вспомогательной сети G_f^L .

Поиск блокирующего потока за $O(|V||E|)$

Блокирующий поток можно найти, запуская поиск в глубину по ненасыщенным рёбрам (т.е. рёбрам (u, v) таким, что $f(u, v) < c(u, v)$) вспомогательной сети G_f^L до тех пор, пока существует $s \rightsquigarrow t$ путь. Такой простой подход будет работать за $O(|E|^2)$ в худшем случае, т.к. каждый обход в глубину (кроме последнего) насыщает хотя бы одно рёбро.

Ясно, что если ребро (u, v) стало насыщенным, то нет смысла явно итерироваться по нему на всех последующих запусках обхода в глубину и вместо этого его нужно явно удалить из списка смежности для вершины u . Также, если обход в глубину прошёл вдоль ребра (u, v) и не нашёл $v \rightsquigarrow t$ пути, то данное ребро также можно удалить из списка смежности, т.к. рёбра в рамках поиска блокирующего потока только удаляются и путь $v \rightsquigarrow t$ не может появиться в рамках следующих запусков обхода в глубину.

Заметим теперь, что если очередной обход в глубину просмотрел e_i рёбер, то после этого хотя бы $e_i - |V|$ рёбер будет удалено из графа, так как удаление касается всех рёбер кроме тех, которые принадлежат $s \rightsquigarrow t$ пути. Тогда, один обход работает за время $O(e_i)$, а так как $\sum_i (e_i - V) \leq |E|$, то $\sum_i e_i \leq |V||E|$ и множество обходов в глубину для нахождения блокирующего потока работает суммарно за время $O(|V||E|)$.

Декомпозиция потока

Докажем важное утверждение о структуре любого потока в транспортной сети, позволяющее декомпозировать его на набор путей и циклов.

Лемма 3. *Для транспортной сети G , любой поток f в ней можно представить в виде декомпозиции из $O(|E|)$ циклов и $s \rightsquigarrow t$ путей. Более формально, существует набор путей $\{p_1, p_2, \dots, p_a\}$ и циклов $\{c_1, c_2, \dots, c_b\}$ такой, что $a + b = O(|E|)$ и $\forall u, v : f(u, v) = \sum_i p_i(u, v) + \sum_i c_i(u, v)$, а величина потока совпадает с суммарной величиной путей $|f| = |\sum_i p_i|$.*

Доказательство. Докажем лемму конструктивно, построив искомую декомпозицию. Если поток из истока s положителен, то существует хотя бы одно ребро (s, v_1) с положительным потоком $f(s, v_1) > 0$. Перейдем по этому ребру и пользуясь свойством сохранения потока найдем следующее ребро (v_1, v_2) где $f(v_1, v_2) > 0$ (оно существует, потому что $f(v_1, s) = -f(s, v_1) < 0$). Будем продолжать эту процедуру до тех пор пока не окажемся в истоке t , либо в уже посещённой вершине. В первом случае можно выделить $s \rightsquigarrow t$ путь, а во втором — цикл с пропускной способностью равной минимальному потоку по соответствующим рёбрам (обратим внимание, что в случае цикла нужно рассматривать только рёбра цикла, исключая возможный предшествующий путь из s). После этого можно уменьшить величину потока вдоль соответствующих рёбер и повторить процедуру заново.

По окончании данного алгоритма будет выделено $O(|E|)$ путей и циклов, а величина потока будет равна 0. Однако, в графе все ещё могут быть рёбра с положительным потоком. Чтобы завершить доказательство достаточно повторить процедуру для всех вершин, откуда выходит хотя бы одно ребро с положительным потоком. В данном случае **не может быть** найден путь до стока (так как величина потока равна 0, а значит сумма входящих потоков в t тоже равна 0), а значит в декомпозицию добавятся только циклы.

Осталось заметить, что выделение каждого пути или цикла насыщает хотя бы одно ребро, а значит суммарно их не может быть больше чем $|E|$.

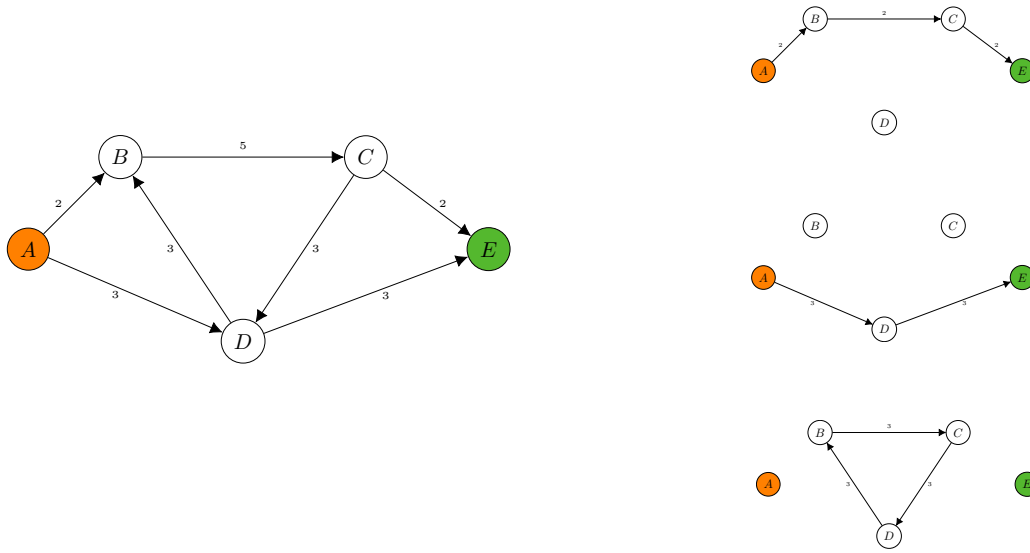


Рис. 3: Декомпозиция в сети на рисунке состоит из двух путей (A, B, C, E) и (A, D, E) с пропускной способностью 2 и 3 соответственно и одного цикла (B, C, D, B) с пропускной способностью 3.

□

Алгоритм Диница на сетях с единичными пропускными способностями

Алгоритм Диница напоминает алгоритм Хопкрофта-Карпа для поиска паросочетания, где на каждой итерации находилось максимальное по включению множество вершинно не пересекающихся чередующихся цепей. Можно заметить, что данный процесс сильно напоминает поиск блокирующего потока в алгоритме Диница.

На самом деле поиск цепей в алгоритме Хопкрофта-Карпа является частным случаем процесса поиска блокирующего потока. Покажем, что алгоритм Диница работает за время $O(|E|\sqrt{|V|})$ для случая единичной сети.

Определение 1. *Единичная сеть (unit network) — транспортная сеть, в которой пропускные способности всех рёбер равны единице, и у любой вершины, кроме истока и стока, либо входящее, либо исходящее ребро единственно.*

Во первых заметим, что для транспортной сети с единичными пропускными способностями, поиск блокирующего потока будет работать за $O(|E|)$, так как все просмотренные в рамках одного обхода рёбра будут удалены из графа, а значит $\sum_i e_i \leq |E|$.

Докажем также более строгую оценку на количество внешних итераций алгоритма Диница для случая единичных сетей. Пусть алгоритм уже произвёл $\sqrt{|V|}$ итераций. Тогда кратчайший $s \rightsquigarrow t$ путь имеет длину хотя бы $\sqrt{|V|}$. Пусть f^* — максимальный поток в транспортной сети, а f — поток построенный к текущей итерации алгоритма Диница. Тогда $f^* - f$ является потоком в остаточной сети G_f , а значит его можно декомпозировать на $O(|E|)$ путей и циклов $\{p_1, p_2, \dots, p_a\}$ и $\{c_1, c_2, \dots, c_b\}$ по лемме 3. Заметим, что в единичной сети все пути декомпозиции не пересекаются по внутренним вершинам, а также имеют единичную пропускную способность. Отсюда следует, что суммарная длина путей $\sum_i |p_i|$ не превосходит $|V|$, а так как $|p_i| \geq \sqrt{|V|}$, то всего таких путей не может быть больше чем $\sqrt{|V|}$, следовательно $|f^*| - |f| \leq \sqrt{|V|}$ и алгоритм совершит ещё не более $\sqrt{|V|}$ итераций.

Таким образом, в случае единичной сети алгоритм Диница совершает не более $2\sqrt{|V|}$ итераций, каждая из которых работает за $O(|E|)$ и суммарно работает за $O(|E|\sqrt{|V|})$.

Реализация алгоритма Диница

В реализации алгоритма можно подметить следующие моменты:

- BFS явно строит временный граф (**layered**) состоящий из рёбер принадлежащим кратчайшим путям. Можно обойтись без этого, если хранить массив `edge_ptr[v]` для каждой вершины и сбрасывать

его после очередной внешней итерации алгоритма, одна код с явным графом получается немного проще

- Чтобы эффективно удалять насыщенные/«бесполезные» рёбра, DFS перебирает рёбра с конца и пользуется тем, что удаление последнего элемента динамического массива выполняется за $O(1)$
- В DFS нет необходимости помечать уже посещённые вершины, так как если обход вышел из какой-то вершины, то он либо нашёл дополняющий путь, либо удалил все рёбра из этой вершины

```
1 struct Edge { int to, flow, capacity; };
2 struct Graph {
3     int n, S, T;
4     vector<Edge> edges;           // we store edges in pairs to simplify flow updates operation
5     vector<vector<int>>> graph; // graph stores indices in edges array instead of Edge structs
6     int dfs(int v, int flow_limit, vector<vector<int>>> &layered) {
7         if (v == T) { return flow_limit; }
8
9         while (!layered[v].empty()) {
10             int idx = layered[v].back();
11             layered[v].pop_back();
12
13             auto e = edges[idx];
14             if (e.capacity - e.flow == 0) { continue; }
15
16             int flow = dfs(e.to, min(flow_limit, e.capacity - e.flow), layered);
17             if (flow == 0) { continue; }
18
19             edges[idx].flow += flow; edges[idx ^ 1].flow -= flow;
20             layered[v].push_back(idx);
21             return flow;
22         }
23         return 0;
24     }
25     bool bfs(vector<vector<int>>> &layered) {
26         auto dist = vector<int>(n);
27         for (int i = 0; i < n; i++) { dist[i] = INF; }
28         auto queue = vector<int> { S };
29         dist[S] = 0;
30
31         for (int i = 0; i < (int)queue.size(); i++) {
32             int v = queue[i];
33             for (auto &idx : graph[v]) {
34                 auto e = edges[idx];
35                 if (e.capacity - e.flow > 0 && dist[e.to] > dist[v] + 1) {
36                     dist[e.to] = dist[v] + 1;
37                     queue.push_back(e.to);
38                 }
39                 if (e.capacity - e.flow > 0 && dist[e.to] == dist[v] + 1) {
40                     layered[v].push_back(idx);
41                 }
42             }
43         }
44         return dist[T] < INF;
45     }
46     int max_flow() {
47         int total_flow = 0;
48         while (true) {
49             auto layered = vector<vector<int>>>(n);
50             if (!bfs(layered)) { break; }
51             int flow;
52             while (flow = dfs(S, MAX_FLOW, layered)) { total_flow += flow; }
53         }
54         return total_flow;
55     }
56 };
```